

Software Development Plan – StudyQuest

Titel des Dokuments:

Software Development Plan – StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.2

Datum:

25. Februar 2026

Author:innen:

- Michael Steer (Scrum Master)
- Luke Engehardt (Product Owner)
- Giuliana Carrano (Developer)
- Paul Strasser (Developer)
- Roman Faber (Developer)

Betreuer / Prüfer:

Sascha Wanninger

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbesch
---------	-------	-------	----------------

0.1	09.10.2025	G. Carrano	Initiale Gliederung und Rollenverteilung entworfen
1.0	17.10.2025	G. Carrano	Erstfassung des Software Development Plans erstellt
1.1	17.10.2025	G. Carrano	Technologie- und Tooling-Stack (VS Code, HTML/CSS/JS) ergänzt
1.2	25.02.2026	M. Steer	Meilensteinplan aktualisiert, Finalversion zur Abgabe vorbereitet

Distribution List

Name	Rolle	
------	-------	--

Sascha Wanninger	Prüfer / Betreuer	
Michael Steer	Scrum Master	
Luke Engehardt	Product Owner	
Giuliana Carrano	Developer	
Paul Strasser	Developer	
Roman Faber	Developer	

1. Vorgehensmodell – Scrum

Beschreibung

Das Projekt StudyQuest wird nach dem agilen Vorgehensmodell Scrum durchgeführt. Die Arbeit wird in Sprints organisiert, in denen inkrementell Anforderungen umgesetzt werden. Kontinuierliches Feedback aus Reviews fließt in die weitere Planung ein.

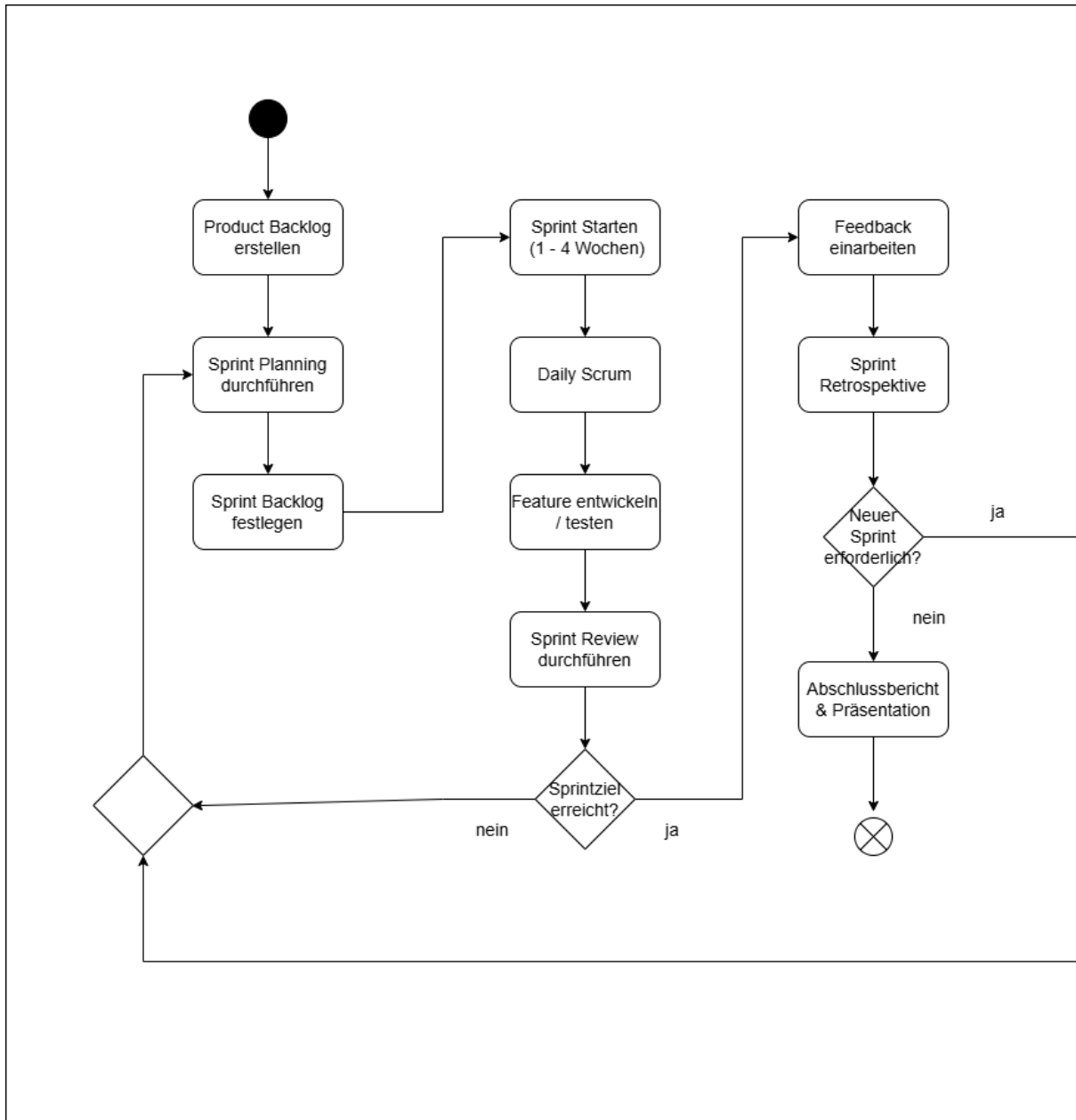
Scrum-Artefakte

- Product Backlog: Priorisierte Anforderungen (User Stories)
- Sprint Backlog: Aufgaben des laufenden Sprints
- Increment: Ergebnis eines Sprints (z. B. funktionsfähiges Mockup oder Dokumentationsteil)

Scrum-Events

- Sprint Planning: Ziele und Umfang des Sprints festlegen
- Daily Scrum: tägliche Kurzabstimmung
- Sprint Review: Ergebnis-Demo und Feedback
- Sprint Retrospektive: Prozessverbesserungen ableiten

2. Aktivitätsbeschreibung



Sprint Planning

Product Owner präsentiert priorisierte User Stories.

Team schätzt den Aufwand (Planning Poker) und plant die Umsetzung.

Scrum Master unterstützt, klärt Abhängigkeiten und erstellt das Sprint Backlog.

Ergebnis: definierte Sprintziele und aktionale Tasks.

Sprint Execution

Team bearbeitet Tasks aus dem Sprint Backlog.

Daily Scrum zur Synchronisation.

Scrum Master beseitigt Impediments.

Sprint Review

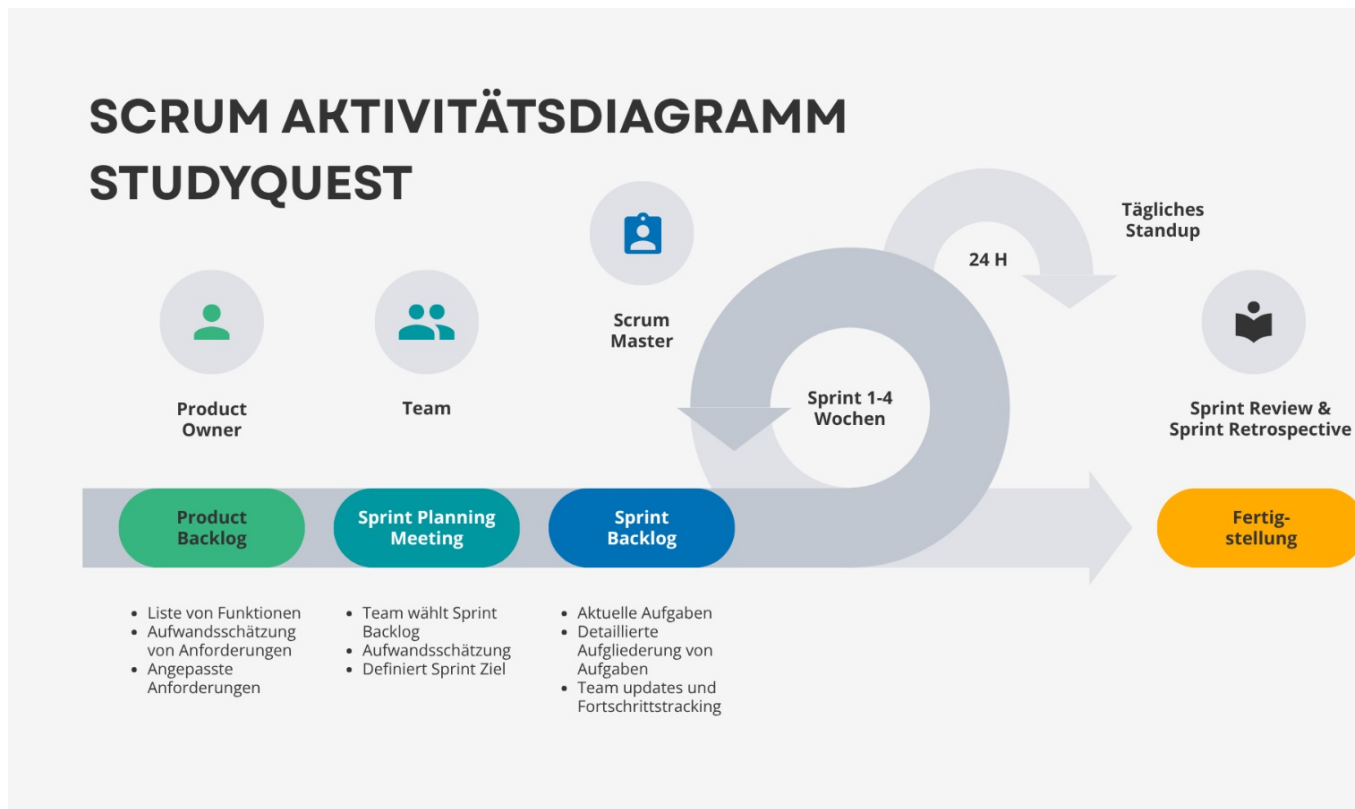
Team demonstriert das Increment.

Feedback durch Stakeholder; Product Backlog wird aktualisiert.

Sprint Retrospektive

Team reflektiert Zusammenarbeit, Tools und Prozess.

Verbesserungsmaßnahmen werden für den nächsten Sprint festgehalten.



3. Teamzusammensetzung

Name	Rolle	Hauptaufgaben
------	-------	---------------

Michael Steer	Scrum Master	Moderation Scrum-Events, Prozessverantwortung, Impediments
Luke Engehardt	Product Owner	Produktvision, Backlog-Pflege, Abnahme
Giuliana Carrano	Developer	Frontend, UI/UX, Dokumentation, Qualitätssicherung
Paul Strasser	Developer	Anwendungslogik, Datenhaltung (leichtgewichtig), Doku
Roman Faber	Developer	Architektur, UML, Integrationstests

4. Verantwortlichkeiten und Aufgabenverteilung

Scrum Master: Einhaltung des Scrum-Prozesses, Organisation der Meetings, Moderation, Beseitigung von Hindernissen.

Product Owner: Definition und Priorisierung der Anforderungen, Pflege des Product Backlogs, Abnahme von Increments.

Developer-Team: Umsetzung der User Stories, Implementierung des Frontends, Anwendungslogik, Erstellung UML/Architektur, Testplanung und Dokumentation.

5. Meilensteine, Reviews und Dokumente

Meilenstein	Geplantes Datum	Ziel / Ergebnis	Dokument / Artefakt
M1 – Projektskizze abgeschlossen	17.10.2025	Projektskizze mit Scope, Risiken, Zielen, Stakeholdern, Glossar und Randbedingungen	Projektskizze.
M2 – Software Development Plan	24.10.2025	Vorgehensmodell, Rollen, Verantwortlichkeiten, Meilensteine, Tooling	SoftwareDevelo
M3 – Anforderungsanalyse	21.11.2025	Funktionale und nicht-funktionale Anforderungen, Use Cases, Systemgrenzen	Requirements.m / UseCases.md
M4 – Grobdesign & Architektur	05.12.2025	Analyseklassenmodell, Sequenzdiagramme, Architektur und Patterns	Analyseklassen / Sequenzdiagram / Softwarearchit

M5 – Prototyp Implementierung	10.01.2026	Kernfunktionen, Gamification, UI, Datenhaltung als lauffähiger Prototyp	src/
M6 – Verifikation & QA	02.02.2026	Testplan, Testfälle, Traceability und Qualitätssicherung	SVP_StudyQuest / SVP_StudyQuest. / SQAP_SQAR_Stud / tests/
M7 – Abschluss & Dokumentation	25.02.2026	Finalisierung der Doku, Reviews, Retrospektive und Abgabe	Software_Design / README.md

Hinweise zur Anwendung im SCRUM-Kontext

- Zwischen den Meilensteinen liegen **Sprints** von ca. 3–4 Wochen Dauer.
- Jeder Meilenstein steht für ein **Review-fähiges Inkrement** (z. B. Analyse, Prototyp, getestete Funktion).

6. Reviews und Retrospektiven

Nach jedem Sprint wird ein Review-Meeting durchgeführt, um den aktuellen Stand zu demonstrieren und Feedback einzuholen. Anschließend findet eine Retrospektive statt, um Prozessverbesserungen für den nächsten Sprint festzuhalten.

7. Dokumentation und Versionskontrolle

Ablage aller Artefakte in einem Git-Repository (z. B. GitHub/GitLab).

Arbeitsweise über Branches und Pull Requests.

Versionierung mittels Tags (z. B. v1.0, v1.1).

Pro Dokument: Changelog und Distribution List pflegen.

8. Technologie- und Tooling-Stack

Editor/IDE

- Visual Studio Code (VS Code)

Web-Technologien

- HTML (Struktur)
- CSS (Layout/Styles)
- JavaScript (Interaktion/Logik, ohne Framework im Prototyp)

Empfohlene VS-Code-Erweiterungen

- Live Server (lokale Vorschau)
- ESLint (JavaScript-Linting)
- EditorConfig (einheitliche Formatierung)
- Prettier (automatische Formatierung)

Code-Qualität und Konventionen

- JavaScript: ES6+, ESLint Standard-Regeln
- CSS: BEM-ähnliche Klassennamen und modulare Struktur (z. B. /styles,

/components)

- Einheitliche Formatierung über Prettier

Zielpattformen (Browser-Support im Prototyp)

- Aktuelle Versionen von Chrome, Edge, Firefox, Safari

Build/Deployment (Prototyp)

- Kein Build-Step erforderlich; statische Auslieferung

Testen (leichtgewichtig)

- Manuelle Funktions- und Usability-Tests anhand definierter Szenarien
- Checklisten für Cross-Browser- und Responsiveness-Tests

9. Aufwandsschätzung

Die Aufwandsschätzung ist Bestandteil der Projektplanung im Scrum-Kontext. Sie erfolgt iterativ auf Basis des Product Backlogs und wird vom gesamten Entwicklungsteam gemeinsam durchgeführt. Zur Schätzung wird eine relative Methode verwendet. Komplexe Aufgaben werden in Schätzkonferenzen diskutiert. Risiken und Unsicherheiten werden bei der Schätzung berücksichtigt und können zu Anpassungen führen. Die Schätzungen werden im Projektverlauf regelmäßig überprüft und bei Bedarf aktualisiert. Die detaillierte Aufwandsschätzung ist im separaten Dokument **Aufwandsschätzung** dokumentiert.

Dokumentationsanforderungen

Alle Dokumente werden im zentralen Repository `studyquest/` abgelegt.

Jedes Dokument enthält: - ein Deckblatt mit Version, Autor und Datum

- einen Änderungsverlauf (Changelog)
- eine Distributionsliste
- Versionskontrolle über Git (Tags, Branches)